

EcoHospital Smart

Ciclo CI/CD com Java Spring Boot

Kalicon Amorim da Cruz Souza — RM 563172

FIAP | Atividade acadêmica | 28/09/2026

Código, testes, containerização e configuração de staging/produção.

CI/CD completo verificado: 25 testes JUnit, imagem GHCR, staging e produção Docker no PC com aprovação obrigatória e mesmo digest. Evidências reais anexadas.

Documento gerado a partir de docs/documentacao-tecnica.md. Outros integrantes: preencher se houver.

1. Identificação e objetivo

Integrante: Kalicon Amorim da Cruz Souza — RM 563172. Instituição identificada no projeto original: FIAP.
Outros integrantes: não informados; preencher se houver. Data da preparação: 28/09/2026.

Objetivo: demonstrar build automático, execução de testes existentes, containerização e promoção da mesma versão em staging e produção. CI, testes, build Docker, publicação GHCR e os dois deploys foram executados e verificados. O usuário escolheu este PC para os dois ambientes, sem servidor pago.

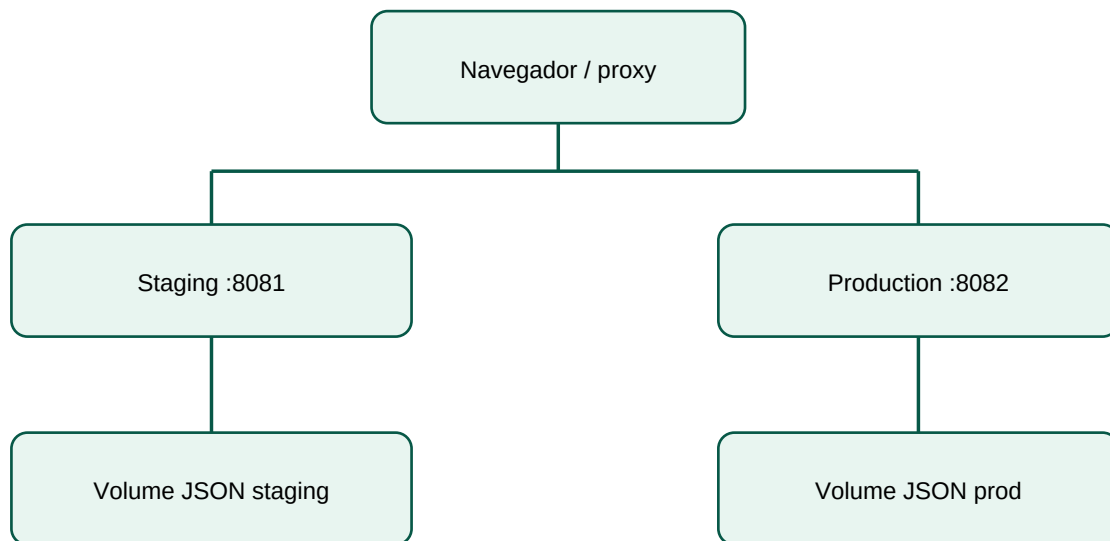
2. Diagnóstico e decisão técnica

O material recebido incluía servidor Node.js nativo, frontend estático, dataset JSON com cinco coleções e scripts acadêmicos MongoDB. Não havia Java/C#, package.json, build, Docker ou CI. A aplicação web simulava consultas MongoDB em arrays e perdia alterações ao reiniciar.

Após autorização para adotar a solução adequada à atividade, foi implementado backend Java 17 com Spring Boot 3.5.16. O frontend e as operações ESG foram mantidos: hospitais, fontes polimórficas, licenças, auditoria, KPIs, consultas, atualizações, simulação IoT e reset. O servidor Node original foi preservado como referência e o runner original continua sendo executado no CI.

Persistência: JSON com gravação atômica em volume por ambiente. Essa escolha mantém o modelo existente com mudanças limitadas e sem serviço de banco artificial. Não se afirma integração MongoDB; seus scripts e prints históricos pertencem à atividade anterior.

3. Arquitetura Docker



Projetos, redes e volumes separados; mesma imagem por digest

Arquitetura prevista: navegador -> proxy/rede do laboratório -> container Spring Boot em cada ambiente -> volume JSON correspondente. O Compose cria uma rede bridge e um volume nomeado por projeto. Staging usa projeto ecohospital-staging e porta 8081; produção usa ecohospital-production e porta 8082. Ambos expõem internamente 8080.

Staging e produção recebem a mesma imagem GHCR por digest SHA256, mas têm configuração e dados separados. Podem estar em dois servidores ou no mesmo host para demonstração, sempre com projetos, portas e volumes distintos. Por padrão, portas no host são limitadas a 127.0.0.1.

O volume contém state.json. O dataset original fica dentro do JAR e é usado somente na criação inicial ou no reset explícito. Reinícios e substituição do container mantêm os dados. Apenas uma instância pode escrever em cada volume. Não há promessa de escalabilidade distribuída, transações MongoDB ou alta disponibilidade.

4. Imagem criada e execução

Dockerfile de dois estágios. Build: maven:3.9-eclipse-temurin-17 executa mvn verify, incluindo JUnit, e gera esg-app.jar. Runtime: eclipse-temurin:17-jre-jammy com JRE, JAR e curl; usuário UID/GID 10001, porta 8080 e healthcheck em /health. Maven e Node não entram no runtime.

O .dockerignore permite apenas pom, Java/testes, frontend e dataset necessário. Compose configura root filesystem somente leitura, tmpfs /tmp, volume de dados gravável, memória de 512 MB, no-new-privileges e capabilities removidas.

Comandos de reprodução: `docker build --build-arg APP_VERSION=academic-local -t ecohospital:local .;`
`docker compose up -d --build --wait --wait-timeout 180.` O pipeline real construiu e testou a imagem; o deploy local usou pull por digest, ``docker compose up --wait`` e health real em 8081/8082.

5. Etapas e lógica do pipeline

Gatilhos: pull request, push na main ou execução manual. verify executa checagem JavaScript, runner histórico com asserções e Maven verify. O JAR e os XML JUnit são guardados como artefatos.

image depende de verify aprovado. Constrói a imagem, executa container, confere usuário, health, versão e persistência após reinício. Na main, publica a imagem testada no GHCR usando GITHUB_TOKEN; o digest real é passado aos dois deploys. Pull requests não publicam nem fazem deploy.

staging depende da imagem e de DEPLOY_ENABLED=true. production depende de imagem e staging e exige PRODUCTION_DEPLOY_ENABLED=true. O Environment production deve ter aprovação configurada no GitHub. Falha em teste, container ou health de staging bloqueia produção pelas dependências needs e pelo código de saída das etapas.

Modo executado: staging-pc e production-pc usam deploy-pc.yml, Docker Desktop Linux e runners Windows efêmeros com label ecohospital-lab. Variables LOCAL_DEPLOY_ENABLED e LOCAL_PRODUCTION_DEPLOY_ENABLED foram habilitadas após o engine ficar pronto. O runner local não executa PRs: exige main e actor Kalicon. Production exigiu e recebeu revisão obrigatória; ambos os Environments estão restritos à main. O pacote GHCR público foi baixado anonimamente por digest, sem PAT permanente. Os runners se removeram após um job cada.

Cada deploy usa secrets do seu Environment: SSH_HOST, SSH_USER, SSH_PORT, SSH_PRIVATE_KEY, SSH_KNOWN_HOSTS, GHCR_USER e GHCR_TOKEN. Variables: APP_PORT, BIND_ADDRESS e APP_URL. A verificação SSH do host é estrita; senha/token não são argumentos de linha de comando.

O servidor faz pull, executa Compose com --wait e confere ambiente e SHA no health. O runner consulta a URL externa e confere os mesmos campos. Evidência JSON é arquivada somente a partir da resposta real. Produção usa o digest de staging sem recompilar. Deploys do mesmo ambiente não ocorrem simultaneamente.

6. Evidências reais disponíveis

Docker Linux 29.8.1 e Compose 5.5.1 responderam neste PC. No run 36438015592, verify, image, staging-pc e production-pc passaram. Kalicon aprovou o Environment production antes do segundo deploy. Ambos usam `ghcr.io/kalicon/ecohospital-esg-cicd@sha256:6a170c3d0f26560afa548a895a67f0f8b8e32481d4a7dc016cc57a7b32b94086` e versão `461ffa3eea37b904018856681c4fbea1cdc2c8b8`. Staging usa porta 8081/volume `ecohospital-staging\_esg-data`; produção usa 8082/volume `ecohospital-production\_esg-data`. Ambos têm health UP e UID 10001. Capturas reais e JSON em docs/evidence/pc.

Teste cruzado real: staging tinha 10 leituras IoT, recebeu uma e ficou com 11; produção permaneceu com 10. Após reiniciar apenas o container staging, manteve 11. JSON `docker-isolation-persistence.json` registra IDs, digest, redes, volumes e contagens. O reinício não apagou nem restaurou volumes.

Duas tentativas anteriores ficaram vermelhas, com produção skipped: primeiro GHCR recusou login do `GITHUB\_TOKEN` para o pacote público; depois DOCKER_CONFIG temporário impediu descoberta do plugin Compose no Windows. A correção foi pull público anônimo sem sobrepor DOCKER_CONFIG. O run de staging aprovado verifica a correção. Essas falhas foram mantidas como evidência do bloqueio de promoção; não foram ocultadas.

Build Maven: aprovado com JDK 17.0.16. JUnit: 25 casos, zero falhas, zero erros e zero ignorados. Runner original: asserções de consultas/CRUD aprovadas para 50 documentos. Logs e XML em docs/evidence/local.

HTTP local: dois processos Java com a mesma versão SHA256 do JAR responderam 200 na página e health. Staging recebeu uma leitura IoT e ficou com 11 leituras; produção permaneceu com 10. Após reinício, staging manteve 11. Esses resultados são reais, porém locais e sem containers.

Actionlint 1.7.12 aprovou os workflows; Git Bash aprovou sintaxe dos scripts SSH. A execução real posterior do Actions aprovou build, smoke test Docker e deploys locais no PC. Não houve deploy SSH porque o usuário escolheu o PC.

Compose standalone 5.5.1 aprovou config dos ambientes sem daemon. As configurações expandidas confirmam portas 8081/8082, redes e volumes com nomes distintos. Esses recursos ainda não foram criados em Docker. A configuração remota foi validada com imagem/digest de exemplo, sem publicação.

CI atualizado aprovado: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/36433415005> . Commit 5b00e1eff60e82993258fdec04263a57fb462e49. Verify e image aprovados; deploys skipped porque ainda desabilitados. Logs, JSON de jobs e captura real em docs/evidence/github.

Imagem GHCR publicada: ghcr.io/kalicon/ecohospital-esg-cicd@sha256:443ba973654cfd1cf9b0992f6f746d0c24eb1ef9b4d4a4562f2580c4eba03a3d . O job image executou container com UID 10001, health UP e ambiente ci; após uma simulação IoT e reinício, preservou 11 leituras no volume. Esses testes ocorreram no runner Ubuntu, não no PC Windows.

Staging real no PC: página e health em <http://localhost:8081>, captura `pc/staging-dashboard.png` e JSON `pc/staging-health.json`. Localhost não é URL pública de hospedagem.

Produção real no PC: aprovação Kalicon registrada em `github/production-approval.json` e captura `pc/github-production-approval.png`; job production-pc aprovado. Página e health em <http://localhost:8082>, captura `pc/production-dashboard.png` e JSON `pc/production-health.json`. Mesmo digest/versão de staging.

Falha proposital comprovada: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/3643347285>. PR 1 / branch demo/test-gate: uma asserção JUnit falhou; image e todos os deploys ficaram skipped. PR encerrado sem merge, preservando a main com testes aprovados. Captura 02-test-gate-failure.png e logs reais arquivados.

7. Desafios e soluções

Linguagem incompatível com o pedido: backend portado para Spring Boot, mantendo contrato de endpoints e dados. Ausência de asserções no runner histórico: adicionadas verificações que geram código não zero em divergência. Perda de dados no reinício: snapshots e substituição atômica de arquivo por ambiente.

Primeiro teste da página falhou porque MockMvc retorna forward sem renderizar o destino. Corrigido para verificar forward e index.html; também executado HTTP real. A execução final dos 25 testes passou.

Ferramentas indisponíveis: Maven resolvido com Wrapper e checksum; JDK 17 existente selecionado. Actions atualizadas após avisos do primeiro run. Docker Desktop oficial instalado com assinatura Authenticode verificada; WSL 2.7.14 habilitado após aprovação UAC e reinicialização manual do usuário. Dois deploys reais foram então executados. Login GHCR recusado e conflito de DOCKER_CONFIG com Compose causaram runs vermelhos; o pacote público passou a ser baixado anonimamente sem modificar configuração do Docker. O CI bloqueou produção nessas falhas.

Interface melhorada: identificação do ambiente e da versão no painel; requisições HTTP com erro não são mais apresentadas como sucesso; contadores não substituem zero por um valor fictício. Funcionalidades ESG e dataset original permanecem preservados.

8. Operação, limitações e entrega

Sem autenticação nas rotas didáticas de mutação/reset; acesso deve ficar limitado ao laboratório ou proxy protegido. Não usar como serviço público de produção sem autenticação e autorização. Google Fonts é externo, com fallback visual do navegador.

Rollback manual preserva volume e usa previous-image.txt do ambiente. Falha de deploy exige diagnóstico; não há rollback automático, zero downtime ou backup remoto automatizado. Dados devem ser copiados antes de mudanças de formato.

Conclusão: Windows reiniciado manualmente, engine Linux verificado, runners efêmeros usados, variables locais habilitadas, staging executado, produção revisada e executada, capturas e logs reais registrados. Não é necessário servidor pago ou SSH para o modo PC. Localhost só funciona enquanto este computador e os containers estiverem ligados. Se a rubrica exigir acesso público externo, essa parte ainda depende de hospedagem/proxy e credenciais próprios; não foi alegada.

ZIP inclui código Java/Node, dados, frontend, Docker/Compose, workflows, Wrapper, scripts, exemplos e documentação/evidências disponíveis; exclui segredos, ferramentas e estados runtime. O manifesto SHA256 permite conferir conteúdo.

9. Referências oficiais

Spring Boot 3.5 — requisitos: <https://docs.spring.io/spring-boot/3.5/system-requirements.html>

GitHub — environments e proteção:

<https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>

GitHub — aprovação e disponibilidade por plano: <https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments/review-deployments>

10. Evolução de segurança, dados e análise ESG

Em revisão posterior à entrega original, foram acrescentadas três melhorias opt-in, sem alterar os volumes existentes. Primeiro, todas as operações que mudam estado — simulação, reset e dois presets update — exigem token de operador por cabeçalho; leitura permanece disponível. O token tem no mínimo 32 caracteres, é comparado por hash em tempo constante e é montado como arquivo secreto no Compose. O painel permite ativar o token apenas na memória da aba e confirma reset explicitamente. Cabeçalhos CSP, no-sniff, no-frame e no-store reduzem a superfície do navegador. O token compartilhado não substitui identidade individual, TLS ou firewall em uma hospedagem pública.

Segundo, `APP_STORAGE=postgres` habilita PostgreSQL 17 com documento JSONB transacional, linha única bloqueada com `SELECT FOR UPDATE` durante escritas e volume persistente sem porta de banco exposta ao host. Na primeira inicialização, o aplicativo importa `state.json` se presente e válido; em reinícios, o estado no banco prevalece. JSON continua sendo o padrão e nenhum volume anterior é apagado. A cópia do staging foi importada em um terceiro Compose isolado em `localhost:8083`: 11 leituras na origem, HTTP 401 sem token, 12 após escrita autenticada e reinício. Essa comprovação é local; não se deve apresentá-la como migração de staging/produção até um deploy validado.

Terceiro, `GET /api/insights` calcula taxa de conformidade, alertas, violações, licenças críticas e ranking de fontes por média de CO2 em relação ao limite cadastrado. A fórmula é mostrada na interface. Os números são descritivos dos registros presentes, não uma certificação ESG nem dados medidos fora da simulação. Em teste local desta revisão, os 30 casos JUnit passaram; o build Docker anterior à inclusão dos três testes extras aprovou 27. O novo smoke test PostgreSQL do CI ainda precisa de execução no GitHub para ser marcado como comprovado.

Anexo — resultados locais reais

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	20	0	0	0
EsgStoreTest	5	0	0	0

HTTP local — staging

```
{"environment": "staging", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "status": "UP"}
```

HTTP local — production

```
{"status": "UP", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "environment": "production"}
```

Verificado em 2026-09-28T10:55:39.2874294-03:00: HTTP 200 em / e /health dos dois processos; versões SHA256 iguais; dados isolados (11/10); persistência após reinício (11). Demonstração local Java, sem Docker ou deploy remoto.

Os resultados acima são da primeira demonstração Java local. As próximas capturas mostram o GitHub Actions e os containers Docker reais no PC.

Anexo — evolução local verificada

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	22	0	0	0
EsgStoreTest	5	0	0	0
WriteAccessTest	3	0	0	0

PostgreSQL isolado no PC: {"checkedAtUtc": "2026-09-28T15:19:55.6166403Z", "environment": "postgres-demo", "storage": "PostgreSQL JSONB", "health": "UP", "readingsBeforeRestart": 12, "unauthorizedResetHttp": 401, "readingsAfterRestart": 12, "databaseContainer": "ecohospital-postgres-demo-db-1", "appContainer": "ecohospital-postgres-demo-app-1"}

Esta prova é do ambiente postgres-demo, não da migração de staging/produção.

Anexo — captura real do GitHub

Run inicial: verify e image aprovados; naquele momento os deploys ainda estavam desabilitados.

The screenshot shows the GitHub Actions interface for the repository 'Kalicon / ecohospital-esg-cicd'. The workflow 'EcoHospital CI/CD' is displayed with a title 'Atualiza Actions e prepara deploy Docker em dois ambientes no PC #2'. The status is 'Success', the total duration is '2m 0s', and there are '3' artifacts. The workflow diagram shows a sequence of steps: 'verify' (green), 'image' (green), 'staging-pc / deploy' (grey), and 'production-pc / deploy' (grey). The 'verify' and 'image' steps are marked as successful, while the deployment steps are greyed out, indicating they are disabled.

Summary

Triggered via push 5 minutes ago

Kalicon pushed [5b00e1e](#) [main](#)

Status	Total duration	Artifacts
Success	2m 0s	3

ci-cd.yml

on: push

```
graph LR; verify[verify] --> image[image]; image --> staging_pc_deploy[staging-pc / deploy]; image --> production_pc_deploy[production-pc / deploy]; staging_pc_deploy --> staging_deploy[staging / deploy]; production_pc_deploy --> production_deploy[production / deploy];
```

Origem: docs/evidence/github/03-updated-ci-success.png. Captura da página real, sem montagem de status.

Anexo — captura real do GitHub

PR descartável: verify falhou; image e deploys bloqueados. PR fechado sem merge.

The screenshot shows a GitHub repository named 'Kalicon / ecohospital-esg-cicd' with a public status. The 'Actions' tab is selected, showing a workflow named 'EcoHospital CI/CD'. The workflow has failed, indicated by a red 'x' icon. The title of the workflow is 'Demonstração acadêmica: testes falhos bloqueiam imagem e deploy #3'. Below the title, it says 'Summary' with a dropdown arrow. The workflow was triggered via a pull request 2 minutes ago. The pull request was opened by 'Kalicon' and is named 'demo/test-gate'. Below this information, there is a table with three columns: 'Status', 'Total duration', and 'Artifacts'. The 'Status' column shows 'Failure', the 'Total duration' column shows '41s', and the 'Artifacts' column shows '1'. Below the table, there is a section titled 'ci-cd.yml' with the trigger 'on: pull_request'. The workflow diagram shows a sequence of steps: 'verify' (failed), 'image', 'staging-pr / deploy', 'production-pr / deploy', 'staging / deploy', and 'production / deploy'. The 'verify' step is marked with a red 'x' and a '17s' duration. The 'image' step is marked with a green checkmark and a '17s' duration. The 'staging-pr / deploy' and 'production-pr / deploy' steps are marked with a green checkmark. The 'staging / deploy' and 'production / deploy' steps are marked with a green checkmark.

Kalicon / ecohospital-esg-cicd Public

<> Code Issues Pull requests **Actions** Projects ...

← EcoHospital CI/CD

✖ Demonstração acadêmica: testes falhos bloqueiam imagem e deploy #3 ...

Summary ▾

Triggered via pull request 2 minutes ago

Kalicon opened #1 [demo/test-gate](#)

Status	Total duration	Artifacts
Failure	<u>41s</u>	<u>1</u>

ci-cd.yml
on: pull_request

```
graph LR; verify[verify 17s] --> image[image 17s]; image --> staging-pr[staging-pr / deploy]; image --> staging[staging / deploy]; staging-pr --> production-pr[production-pr / deploy]; staging --> production[production / deploy];
```

Origem: docs/evidence/github/02-test-gate-failure.png. Captura da página real, sem montagem de status.

Anexo — ambiente Docker no PC

Run final aprovado: verify, image, staging-pc e production-pc.

The screenshot shows a GitHub Actions workflow run for the repository 'Kalicon / ecohospital-esg-cicd'. The workflow is named 'Preserva descoberta do plugin Compose no runner' and is running on a Windows environment. The run is successful, with a status of 'Success' and a total duration of '10m 8s'. The workflow consists of five steps: 'verify', 'image', 'staging-pc / deploy', 'production-pc / deploy', and 'staging / deploy'. The 'verify' and 'image' steps are completed, while the 'staging-pc / deploy' and 'production-pc / deploy' steps are currently running. The 'staging / deploy' and 'production / deploy' steps are not visible in the current view.

← EcoHospital CI/CD

✓ Preserva descoberta do plugin Compose no runner
Windows #9

Summary ▾

Triggered via push 10 minutes ago

Kalicon pushed 461ffa3 main

Status	Total duration	Artifacts
Success	10m 8s	5

ci-cd.yml
on: push

```
graph LR; verify[verify] --> image[image]; image --> staging-pc[staging-pc / deploy]; image --> production-pc[production-pc / deploy]; staging-pc --> staging[staging / deploy]; production-pc --> production[production / deploy];
```

Origem: docs/evidence/pc/github-two-deploys-approved.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

Histórico real: Kalicon aprovou production antes do deploy.

The screenshot displays a deployment interface. At the top, there are two buttons: "staging / deploy" and "production / deploy". Below these, there is a section titled "Deployment protection rules" with the subtitle "Reviewers, timers, and other rules protecting deployments in this run". A table follows with two columns: "Event" and "Environments". The "Event" column shows a user profile for "Kalicon" with a green checkmark and the text "approved 1 minute ago". The "Environments" column shows "production". Below this table is a section titled "Artifacts" with the subtitle "Produced during runtime". A table lists artifacts with columns "Name", "Size", and "Digest". The first artifact is "Kalicon~ecohospital-esg-cicd~FG1XSG.dockerbuild" with a size of "41.2 KB" and a digest of "sha256:a0691750ca123...". At the bottom, there is a section titled "build-and-tests-".

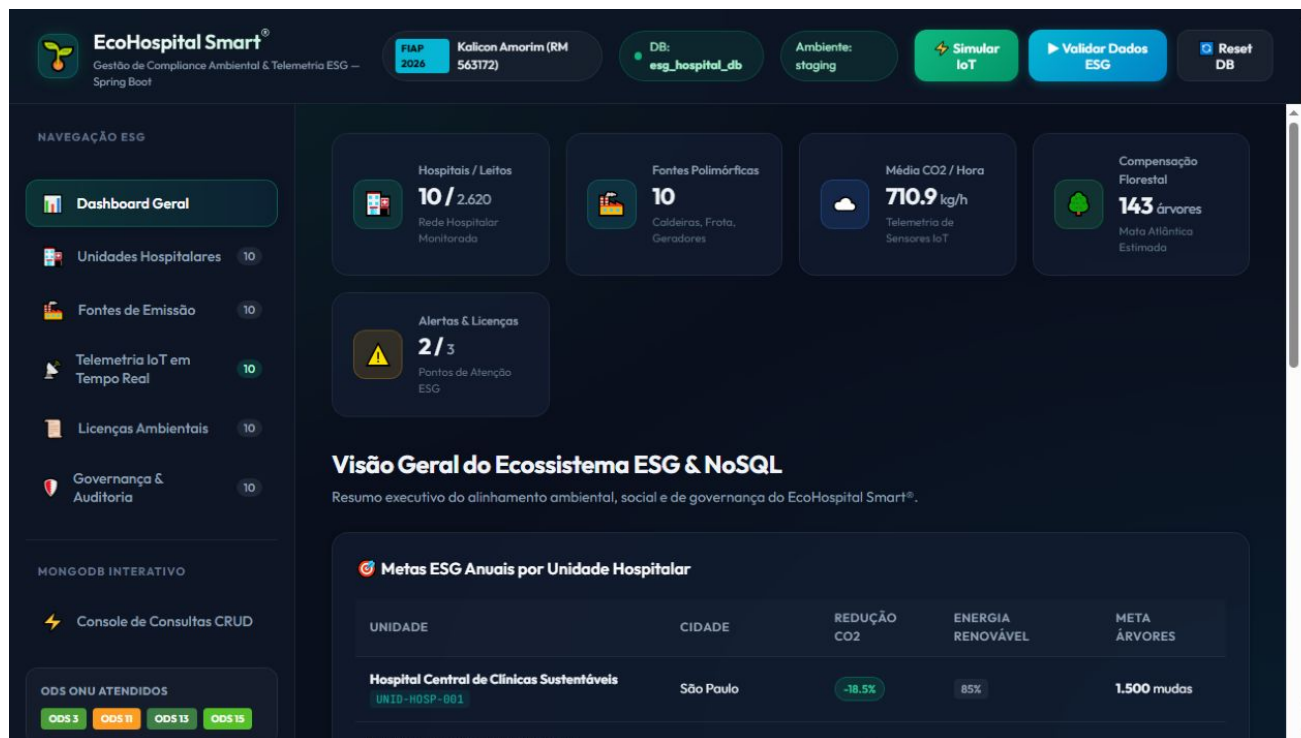
Event	Environments
 Kalicon approved 1 minute ago	production

Name	Size	Digest
 Kalicon~ecohospital-esg-cicd~FG1XSG.dockerbuild	41.2 KB	sha256:a0691750ca123... 

Origem: docs/evidence/pc/github-production-approval.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

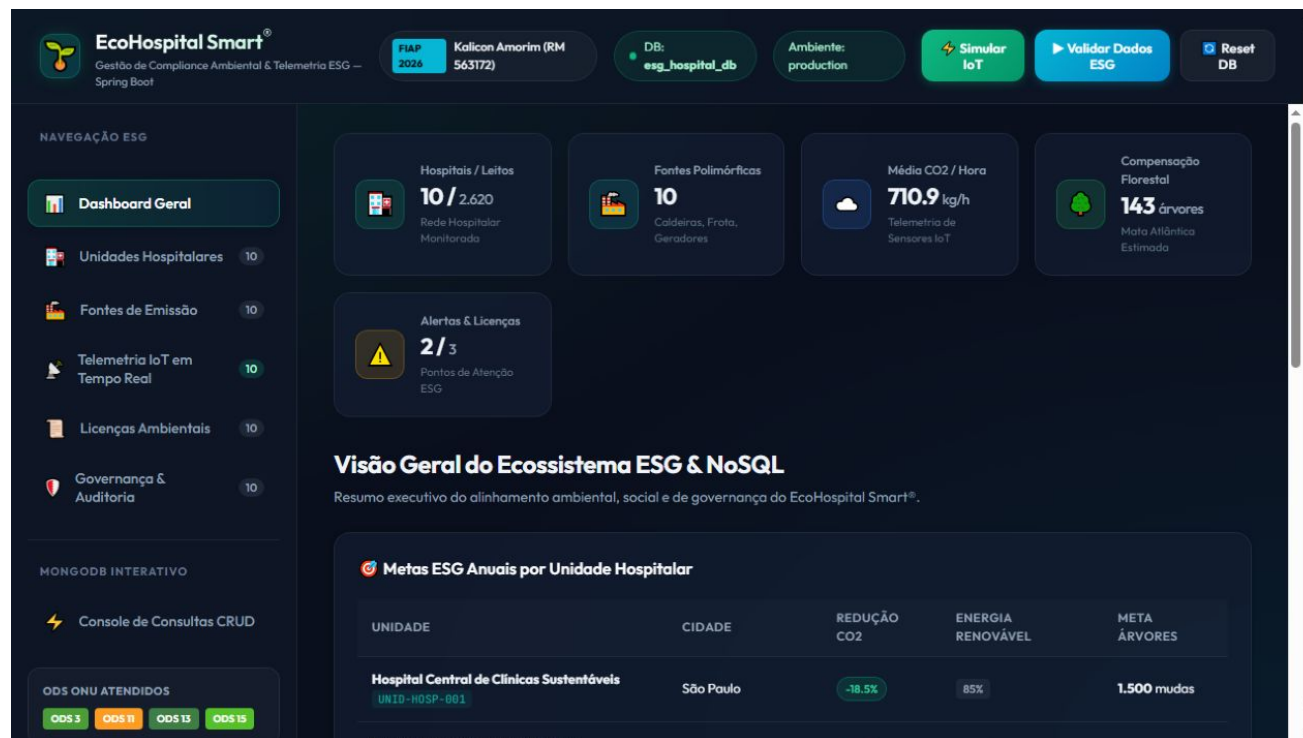
Dashboard real de staging no container Docker deste PC, porta 8081.



Origem: docs/evidence/pc/staging-dashboard.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

Dashboard real de produção no container Docker deste PC, porta 8082.



Origem: docs/evidence/pc/production-dashboard.png. Captura real; URLs localhost não são servidores públicos.